

개발 환경 및 프레임워크

코드 편집기 - Visual Studio Code

1. OpenAI GPT API

용도: OpenAI GPT를 사용하여 주가 예측에 필요한 데이터 임베딩(벡터화) 작업을 수행

설명: OpenAI GPT API는 대규모 언어 모델을 활용하여 텍스트 데이터를 벡터로 변환하고, 이를 기반으로 다양한 자연어 처리 작업을 수행

관련 라이브러리: openai

2. Pinecone

용도: 임베딩된 벡터 데이터를 저장하고, 검색할 수 있는 벡터 데이터베이스

설명: Pinecone은 고차원 벡터 데이터를 저장하고, 유사도 검색 등을 수행할 수 있는 벡터 DB로, NLP나 LLM 기반 시스템에서 임베딩 데이터를 효율적으로 관리

관련 라이브러리: pinecone-client

3. SQLAlchemy

용도: 관계형 데이터베이스(RDBMS)에 임베딩 데이터를 저장하고 관리하기 위한 ORM(Object Relational Mapping) 도구

설명: SQLAlchemy는 데이터베이스의 테이블을 파이썬 객체로 매핑하여, SQL 쿼리 없이도 데이터를 삽입하고 조회할 수 있게 도와줌

Pinecone에서 생성된 벡터 데이터를 MySQL 같은 관계형 DB에 저장하는 데 사용

관련 라이브러리: sqlalchemy

1. 파이썬을 통해 OpenAI GPT API와 연결하여 임베딩 작업

1.1. 사전 준비

- VSC를 설치한 후, 파이썬 환경 설정 완료하기
- 터미널을 통해 필수 라이브러리를 설치(pip install openai pinecone-client)

1.2. OpenAI API 설정

- 발급 받은 OpenAI API 키 입력
- 텍스트 임베딩하기 위한 GPT-3의 임베딩 모델 사용

1.3. 산출물

- 임베딩 생성 코드 파일

OpenAI GPT를 통해 주어진 텍스트 데이터를 벡터화하는 과정이 포함된 코드 파일

2. 임베딩 데이터를 Pinecone 벡터 DB에 정리

2.1. 파인콘 초기화 및 API 키 설정

2.2. 임베딩된 데이터를 Pinecone에 업로드

2.3. 산출물

- 임베딩 저장 코드 파일

OpenAI GPT에서 생성된 임베딩을 Pinecone 벡터 DB에 저장하는 코드 파일

3. 임베딩된 데이터를 다시 DB에 정리

3.1. 데이터베이스에 정리하기 위해, MySQL DB에 연결

- SQLAlchemy 라이브러리 사용

SQLAlchemy는 파이썬에서 데이터베이스와 상호작용할 수 있게 해주는 ORM(Object Relational Mapping) 라이브러리로, SQL 쿼리를 직접 작성하지 않아도 객체 지향적으로 데이터를 관리할 수 있게 도와준다. 즉, Pinecone에서 임베딩 데이터를 처리한 후, 쉽게 SQL 데이터베이스에 저장할 수 있다.

3.2. DB 스키마 설정 및 연결

3.3 임베딩 데이터를 DB에 저장

- 임베딩을 문자열로 변환 (JSON 형식으로 저장)

3.4. 산출물

- DB 저장 코드 파일

OpenAI GPT 임베딩된 데이터를 SQL DB에 정리하는 코드 파일

-> SQLAlchemy를 사용해 MySQL 등의 DB와 연결하고 데이터를 저장

4. NLP에 대해 구현 방법 조사 및 LlamaIndex 사용

4.1. LlamaIndex 구현 조사

- LlamaIndex는 다양한 벡터 및 자연어 검색 기능을 제공하는 툴로, 데이터베이스나 문서에서 효율적으로 검색을 수행

- 텍스트 전처리: 불필요한 단어 제거, 토큰화, 벡터화 처리.

- LlamaIndex를 활용한 텍스트 분류: 뉴스 기사나 주가 관련 정보를 효율적으로 검색하거나 분류하는 알고리즘

- 코사인 유사도 계산: 각 임베딩 간 유사도를 계산하여, 주가 예측과 관련된 추천 시스템을 구현

4.2. 산출물

- NLP 전처리 및 LlamaIndex 코드 파일

LlamaIndex를 사용한 벡터 검색 및 NLP 기능을 구현하는 코드

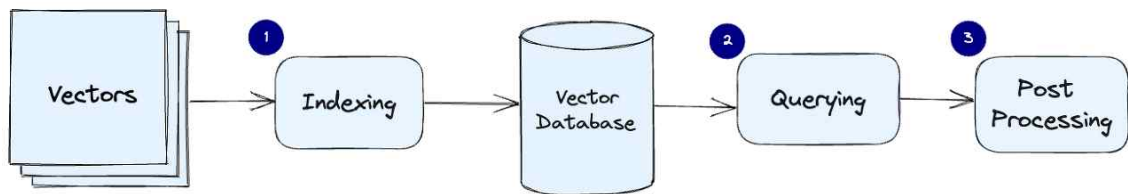
-> LlamaIndex를 이용해 임베딩된 데이터를 검색하고 처리.

<벡터 데이터베이스 동작 방식>

관계형 데이터베이스는 쿼리와 정확히 일치하는 행을 찾아 반환

벡터 데이터베이스에서는 유사도 메트릭(Similarity Metrics)을 적용하여 가장 유사한 벡터를 찾음

벡터 데이터베이스의 일반적인 파이프라인



1. 색인(Indexing) : PQ, LSH, HNSW와 같은 알고리즘을 사용하여 벡터를 색인
이 단계에서는 더 빠른 검색을 가능하게 하는 데이터 구조에 벡터를 매핑함

2. 쿼리 : 유사도 메트릭(코사인 유사도, 유클리디안 거리, 내적 등)을 적용하여 가장 가까운 벡터를 찾음

코사인 유사도(Cosine Similarity) : 두 벡터 간의 각도를 측정하여 유사성을 판단합니다. 두 벡터가 얼마나 같은 방향으로 향하고 있는지를 나타냄
값은 -1 ~ 1 사이에 있으며, 값이 1에 가까울수록 두 벡터는 유사하고, -1에 가까울수록 반대 방향에 있는 것을 나타냄

3. 후처리 : 데이터셋에서 최종 최근접 이웃을 검색하고 이를 후처리하여 최종 결과를 반환함
일부 벡터 데이터베이스는 벡터 검색을 하기 전에 필터를 적용할 수 있음